

Relazione del progetto di Ingegneria del Software 2

Defect prediction, what-if analysis e refactoring automatico

Progetto Apache: **SYNCOPE**

Leonardo Polidori – Matricola: 0357314

[Repository GitHub](#)
[Dashboard SonarCloud](#)

Anno Accademico 2025/2026

Sommario

La relazione presenta uno studio empirico sul progetto Apache **Syncope**, finalizzato ad analizzare la relazione tra difetti, metriche software e code smell. Il lavoro comprende la costruzione di un dataset di defect prediction, la valutazione di modelli predittivi, una what-if analysis sull'assenza di smell e una fase finale di refactoring automatico tramite LLM.

Indice

1	Introduzione	2
2	Metodologia	2
2.1	Milestone 1: Creazione del dataset	2
2.2	Milestone 2: Valutazione dei classificatori	3
2.3	Milestone 3: What-if analysis	4
2.4	Milestone 4: Refactoring automatico	5
3	Risultati	6
3.1	RQ1 – Come si caratterizza il dataset costruito per Syncope?	6
3.2	RQ2 – Quale classificatore risulta più accurato?	6
3.3	RQ3 – Quante classi buggy si sarebbero potute prevenire con zero smell?	7
3.4	RQ4 – Quanto è accurato ed efficace il refactoring automatico tramite LLM?	8
4	Minacce alla validità	8
4.1	Minacce relative alla Milestone 1	8
4.2	Minacce relative alla Milestone 2	9
4.3	Minacce relative alla Milestone 3	10
4.4	Minacce relative alla Milestone 4	10
5	Conclusioni e takeaway messages	11
	Tabelle e figure	12

1 Introduzione

Apache Syncope è un progetto open-source della Apache Software Foundation dedicato alla gestione delle identità digitali, dell'access management e del provisioning. Il progetto è stato organizzato in quattro milestone, ciascuna dedicata a una fase diversa dell'analisi di Apache Syncope. La Milestone 1 costruisce un dataset di defect prediction a livello classe-release, integrando metriche software, code smell e label di bugginess. La Milestone 2 confronta diversi classificatori, considerando feature selection e balancing. La Milestone 3 applica una what-if analysis per stimare l'effetto di uno scenario ipotetico con zero code smell sulle predizioni di bugginess. La Milestone 4 valuta il refactoring automatico tramite LLM su due classi dell'ultima release del progetto.

L'obiettivo del lavoro è valutare se le metriche raccolte sulle classi di Syncope permettano di prevedere la bugginess e, successivamente, se la rimozione degli smell possa modificare tali predizioni senza compromettere il comportamento del codice.

2 Metodologia

2.1 Milestone 1: Creazione del dataset

La Milestone 1 ha prodotto il dataset usato come base per le analisi successive. Il dataset è definito a livello classe-release: ogni istanza rappresenta l'osservazione di una classe Java in una specifica release, identificata dalla coppia $\langle \text{ClassPath}, \text{ReleaseID} \rangle$, e associa a tale osservazione metriche software, informazioni sugli smell e la label finale **Buggy**, con valori *yes/no*.

Il processo comprende la selezione delle release, la costruzione dell'inventario, la gestione dei ticket, l'individuazione dei fix commit, l'identificazione delle classi buggy, il calcolo degli smell, il calcolo delle metriche e il merge finale.

2.1.1 Release e istanze classe-release

Le release sono state lette dal file `SYNCOPEVersionInfo.csv`, disponibile nel repository GitHub del progetto, e ordinate secondo la data e l'indice associato alla release.

In accordo con la milestone, è stato considerato il primo 34% delle release, scartando il restante 66%. Per ogni release selezionata è stato individuato lo snapshot Git corrispondente, scegliendo l'ultimo commit disponibile entro la data della release; operativamente, il limite temporale è stato posto al giorno successivo alla release come limite esclusivo.

Per ciascuno snapshot sono state individuate le classi Java di produzione, escludendo i file di test. L'inventario di ciascuna release è stata usata per assegnare label positive solo a classi effettivamente presenti nello snapshot corrispondente.

2.1.2 Ticket, versioni affette e Proportion Total

I ticket sono stati letti da `SYNCOPETickets.csv`, considerando validi i difetti chiusi o risolti con risoluzione *Fixed*; il filtro al primo 34% è stato applicato alle release del dataset finale, non alla lettura dei ticket. Poiché la label è assegnata a livello classe-release, le informazioni temporali dei ticket sono state trasformate in un intervallo di release affette.

Per ogni ticket sono state ricostruite tre versioni: **OV** (*Opening Version*), cioè la release più recente alla data di apertura del ticket; **FV** (*Fixed Version*), cioè la prima release successiva alla data di risoluzione; e **IV** (*Injected Version*), cioè la release di introduzione del difetto, ricavata dalle *Affected Version* quando disponibili e coerenti.

La IV ricavata dalle *Affected Version* è stata accettata solo se presente nella lista delle release, non successiva alla OV e strettamente precedente alla FV. Negli altri casi, la IV è stata stimata tramite **Proportion Total**, usando la proporzione media osservata nei ticket con AV valida. La

IV stimata è stata infine arrotondata alla release più vicina e vincolata a non essere successiva alla OV.

La **ComputedAV** è stata quindi definita come intervallo $[IV, FV)$, con IV inclusa e FV esclusa. Poiché non era disponibile la data effettiva del fix commit, la *resolution date* del ticket è stata usata come proxy temporale per la FV.

2.1.3 Fix commit, SZZ e classi buggy

Per collegare i ticket ai cambiamenti nel codice, sono stati cercati nel repository Git i commit contenenti gli identificativi dei ticket. A partire da tali fix commit sono state individuate le classi Java di produzione coinvolte nella correzione tramite una procedura ispirata a SZZ.

Per ogni fix commit è stato analizzato il diff rispetto al parent commit e, per il codice già presente nel parent, è stato eseguito `git blame`. In questa pipeline, SZZ è stato usato per individuare le classi associate ai ticket di bug; la collocazione temporale della bugginess è stata invece gestita tramite la **ComputedAV**.

Una classe è stata marcata come buggy in una release solo se la release apparteneva alla **ComputedAV** del ticket e se la coppia $\langle \text{ClassPath}, \text{ReleaseID} \rangle$ era presente nell'inventario dello snapshot. Questo vincolo evita di propagare label positive a release in cui la classe non era presente.

2.1.4 Smell, metriche e dataset finale

Gli smell sono stati calcolati per ogni coppia classe-release utilizzando **PMD** con il ruleset `rulesets/java/quickstart.xml`. In questa relazione, uno smell corrisponde operativamente a una violation PMD rilevata dal ruleset adottato. Il numero di violation è stato usato per valorizzare **NSMELLS**; inoltre, è stata calcolata anche **SMELL_DENSITY**, ottenuta normalizzando il numero di smell rispetto alla dimensione della classe.

Per ogni coppia classe-release sono state inoltre calcolate metriche predittive strutturali, evolutive, di fix history, ownership, contesto dei commit e smell. Le metriche storiche sono state calcolate usando solo la storia Git raggiungibile dallo snapshot della release corrente, così da rispettare il realismo temporale dell'esperimento. Il set completo delle metriche, con finestra temporale e significato sintetico, è riportato nella Tabella 1.

Il dataset finale combina la label **Buggy**, con valori *yes/no*, le metriche software e le informazioni sugli smell in un'unica rappresentazione classe-release, usata come base per le milestone successive. Prima dell'uso sono stati controllati duplicati, esclusione delle classi di test, coerenza delle metriche e corretta associazione tra smell, metriche e label.

2.2 Milestone 2: Valutazione dei classificatori

L'obiettivo della Milestone 2 è stato valutare la capacità predittiva di diversi classificatori sul dataset prodotto nella Milestone 1. Il problema è stato formulato come classificazione binaria, usando **Buggy** come classe target e **yes** come classe positiva, cioè la classe buggy.

Gli esperimenti sono stati eseguiti in WEKA, confrontando più classificatori, filtri e configurazioni sullo stesso dataset. Poiché la classe buggy è minoritaria, la valutazione non si è basata sulla sola correttezza complessiva, ma su metriche orientate alla capacità di individuare le classi difettose.

2.2.1 Preparazione del dataset per WEKA

Il dataset finale della Milestone 1 è stato convertito in formato ARFF per l'esecuzione degli esperimenti in WEKA. Prima della conversione sono state rimosse le colonne identificative, come progetto, release e percorso della classe, mantenendo solo le feature numeriche calcolate per ogni coppia classe-release.

Nel file ARFF l'attributo `Buggy` è stato dichiarato con ordine `{yes,no}`, così da interpretare le metriche di WEKA rispetto alla classe positiva `yes`.

2.2.2 Classificatori e validazione

Sono stati valutati i tre classificatori richiesti dalla Milestone 2: `RandomForest`, `NaiveBayes` e `IBk`.

La valutazione è stata eseguita tramite *10 times 10-fold cross-validation*. Ogni configurazione è stata quindi valutata su 10 run da 10 fold, per un totale di 100 valutazioni. Le prestazioni finali sono state aggregate calcolando i valori medi delle metriche sui fold e sulle run, riducendo la dipendenza da una singola partizione del dataset.

2.2.3 Feature selection e balancing

La feature selection è stata applicata in WEKA tramite `InfoGainAttributeEval` e `Ranker`. Questa configurazione produce un ranking degli attributi in base all'Information Gain rispetto alla classe target, senza eseguire una ricerca bottom-up o top-down di sottoinsiemi di feature.

Sono stati valutati tre tagli del ranking: top 5, top 10 e top 15 feature, così da confrontare soglie più o meno aggressive di selezione. Poiché il dataset è sbilanciato verso la classe non buggy, è stato applicato anche il filtro `ClassBalancer`, che riequilibra il contributo delle classi modificando i pesi delle istanze, senza duplicare istanze buggy né rimuovere istanze della classe maggioritaria.

Nei casi in cui feature selection e balancing sono stati applicati congiuntamente, la feature selection è stata eseguita prima del bilanciamento. I filtri sono stati incapsulati nella meta-classe `FilteredClassifier` di WEKA, così da applicare selezione delle feature e bilanciamento solo sui training fold durante la cross-validation ed evitare data leakage.

2.2.4 Configurazioni sperimentali e metriche

Gli esperimenti sono stati organizzati in configurazioni progressive: baseline senza filtri, solo balancing, solo feature selection e combinazione di feature selection e balancing con top 5, top 10 e top 15 feature. Il confronto tra queste configurazioni ha permesso di valutare sia l'effetto dei singoli filtri sia quello della loro applicazione congiunta.

Le prestazioni sono state valutate tramite Precision, Recall, AUC e Kappa, tutte riferite alla classe positiva `yes`. Precision misura l'affidabilità delle predizioni buggy, Recall la capacità di individuare le classi buggy reali, AUC la separabilità tra le due classi e Kappa l'accordo tra predizioni e label al netto dell'accordo casuale. La sola accuracy non è stata usata come criterio principale, perché in un dataset sbilanciato può favorire modelli orientati alla classe maggioritaria `no`.

2.3 Milestone 3: What-if analysis

La Milestone 3 ha stimato quante istanze classificate come buggy nello scenario reale sarebbero state classificate come non buggy in uno scenario ipotetico con zero code smell.

L'analisi è stata impostata come una *what-if analysis*, intervenendo sulle feature `NSMELLS` e `SMELL_DENSITY`. Entrambe sono feature *actionable*, poiché potenzialmente riducibili tramite refactoring.

Le predizioni sono state ottenute con il classificatore selezionato nella Milestone 2, cioè `RandomForest` con `ClassBalancer`. Lo scenario sintetico misura l'effetto sulle predizioni del modello, senza dimostrare una relazione causale certa tra smell e difetti.

2.3.1 Dataset della what-if analysis

A partire dal dataset finale della Milestone 1 sono stati costruiti quattro dataset. **A** è il dataset originale, contenente tutte le istanze classe-release. **B+** è il sottoinsieme di **A** con almeno uno smell ($\text{NSMELLS} > 0$). **C** è il sottoinsieme di **A** già privo di smell ($\text{NSMELLS} = 0$). Infine, **B** è la versione sintetica di **B+**, ottenuta forzando $\text{NSMELLS} = 0$ e $\text{SMELL_DENSITY} = 0$.

Tutte le altre feature sono state mantenute invariate.

2.3.2 Applicazione del classificatore e stima dei prevenibili

Il classificatore è stato addestrato sull'intero dataset **A**. In questa fase non è stata usata la cross-validation, perché l'obiettivo non era stimare nuovamente l'accuratezza del modello, ma applicare il modello già selezionato a scenari alternativi. Le predizioni sono state quindi calcolate sui dataset derivati tramite l'opzione *supplied test set* di WEKA.

Il confronto è tra **B+** e **B**, che contengono le stesse istanze ma differiscono per le feature di smell. Sono considerate potenzialmente prevenibili le istanze predette buggy in **B+** e non buggy in **B**. La misura è quindi una stima controfattuale basata sul modello, non una prova causale che la rimozione degli smell avrebbe certamente eliminato un bug.

2.4 Milestone 4: Refactoring automatico

La Milestone 4 ha valutato il refactoring automatico tramite LLM su classi di Apache Syncope. Per ciascuna classe selezionata, la versione originale C_0 è stata usata come baseline per generare quattro varianti rifattorizzate, indicate come C_1 – C_4 , fornendo al modello quantità crescenti di contesto sperimentale.

La selezione delle classi è stata effettuata sull'ultima release disponibile, 4.1.0.

2.4.1 Selezione delle classi

Le classi Java di produzione dell'ultima release sono state analizzate tramite PMD e ordinate in base a NSMELLS . Il ranking è stato poi filtrato per escludere classi troppo semplici, generate, di test o poco adatte alla validazione tramite test, usando soglie su dimensione, numero di metodi e complessità ciclomatica.

La selezione finale ha prodotto due classi concrete: `DefaultJobManager` e `AuthDataAccessor`.

2.4.2 Generazione delle varianti tramite LLM

Per ciascuna classe selezionata, la versione originale C_0 è stata usata come baseline per generare quattro varianti rifattorizzate, indicate come C_1 – C_4 . Le varianti sono state prodotte con **Microsoft Copilot**, usando la modalità *Thinking*.

Il prompt richiedeva di migliorare la manutenibilità e ridurre gli smell segnalati da SonarCloud, senza modificare il comportamento funzionale della classe. La variante generata doveva poter sostituire C_0 nel progetto, mantenendo package, nome della classe, API pubblica e firme dei metodi pubblici e protetti. Il prompt base è riportato nel Listato 1.

La generazione ha seguito una progressione incrementale di contesto: C_1 usa la classe originale e la diagnostica SonarCloud; C_2 aggiunge i test black-box; C_3 aggiunge i test control-flow; C_4 aggiunge i test derivati dal mutation testing. In questo modo è stato possibile osservare se un maggiore contesto sperimentale portasse a varianti più conservative e aderenti al comportamento di C_0 .

2.4.3 Valutazione delle varianti

Dopo la generazione, ogni variante C_X è stata copiata temporaneamente nel path originale della classe corrispondente, così da compilare ed eseguire il progetto come se la variante fosse la reale sostituta di C_0 .

Le varianti sono state valutate secondo tre criteri: compilazione nel progetto, riduzione delle issue statiche e preservazione del comportamento osservabile tramite riesecuzione delle suite di regressione costruite su C_0 . Il superamento dei test non dimostra equivalenza funzionale completa, ma indica che i comportamenti coperti dagli oracoli disponibili non sono stati alterati.

3 Risultati

3.1 RQ1 – Come si caratterizza il dataset costruito per Syncope?

La RQ1 descrive la composizione del dataset costruito nella Milestone 1. Il dataset finale contiene 20412 istanze classe-release, ciascuna relativa a una classe Java di produzione osservata in una specifica release di Apache Syncope.

La distribuzione delle istanze mostra un forte sbilanciamento verso la classe non buggy: 1155 istanze sono etichettate come buggy, mentre 19257 sono non buggy, per una percentuale complessiva di bugginess pari al 5.66% (Figura 1).

A livello di release, il numero di classi cresce in modo complessivo ma non lineare: si passa da 488 classi nella prima release considerata a 1310 classi nella release 2.0.0-M1, con un picco di 1462 classi nella release 1.2.6. La percentuale di classi buggy varia tra le release e raggiunge il massimo nella release 1.0.5, con circa il 21.19%. Il dettaglio della distribuzione per release è riportato nella Tabella 2.

Per quanto riguarda gli smell e il linkage Jira-Git, gli indicatori principali sono riportati nella Tabella 3. Il dato conferma che una parte rilevante delle istanze presenta almeno uno smell e che il collegamento tra ticket e fix commit è elevato.

Il confronto descrittivo tra classi buggy e non buggy è riportato nella Tabella 4. Le classi buggy risultano più grandi, più complesse e più modificate: in particolare, mostrano valori mediani più elevati su metriche di dimensione, complessità e churn.

La Figura 2 riporta la correlazione di Spearman tra le feature numeriche e la label Buggy. L'asterisco indica le correlazioni statisticamente significative con $p\text{-value} < 0.05$. Poiché il dataset contiene un numero elevato di istanze, anche correlazioni di intensità ridotta possono risultare statisticamente significative. Per questo motivo, le correlazioni sono interpretate considerando sia il $p\text{-value}$ sia il valore assoluto del coefficiente di Spearman. Le associazioni più marcate riguardano FAN_OUT, SIZE_LOC, metriche di churn e complessità ciclomatica, mentre NSMELLS e SMELL_DENSITY mostrano un'associazione positiva ma più debole.

3.2 RQ2 – Quale classificatore risulta più accurato?

La RQ2 confronta RandomForest, NaiveBayes e IBk sul dataset costruito nella Milestone 1, valutando l'effetto di feature selection e balancing. La valutazione è stata effettuata tramite 10 times 10-fold cross-validation, usando Precision, Recall, AUC e Kappa rispetto alla classe positiva yes. I risultati sono riportati nelle Tabelle 5 e 6.

La migliore configurazione complessiva è RandomForest con ClassBalancer. Questa configurazione ottiene Precision pari a 0.611, Recall pari a 0.526, AUC pari a 0.914 e Kappa pari a 0.540. La scelta è motivata dal miglior compromesso tra capacità di individuare le istanze buggy e affidabilità delle predizioni positive: rispetto alla baseline senza filtri, il balancing aumenta la Recall sulla classe yes, mantenendo AUC e Kappa su valori elevati. È stata quindi selezionata come modello di riferimento per la Milestone 3.

Il confronto con le altre configurazioni mostra che RandomForest resta il classificatore più solido anche quando feature selection e balancing vengono applicati congiuntamente. Tra le configurazioni con entrambi i filtri attivi, il risultato migliore è ottenuto da RandomForest con top 15 feature, con Precision pari a 0.588, Recall pari a 0.513, AUC pari a 0.908 e Kappa pari a 0.522.

IBk rappresenta generalmente il secondo classificatore più competitivo, mentre NaiveBayes risulta meno efficace, soprattutto a causa della bassa Precision sulla classe buggy.

Questo comportamento è coerente con la natura del dataset: come mostrato nella Figura 3, molte metriche software risultano correlate tra loro, rendendo meno adatta l'assunzione di indipendenza alla base di NaiveBayes.

L'analisi delle soglie di feature selection mostra che top 5 è troppo aggressivo per RandomForest, mentre top 10 e top 15 mantengono più informazione predittiva. Come mostrato nella Figura 4, il passaggio da top 5 a top 15 porta Precision da 0.438 a 0.588, AUC da 0.873 a 0.908 e Kappa da 0.433 a 0.522. Questo suggerisce che la difettosità non è spiegata solo dalle prime feature del ranking, ma da una combinazione di dimensione, complessità, churn, fix history, ownership e smell.

Nel complesso, la feature selection consente di ridurre il numero di metriche mantenendo prestazioni vicine al modello migliore; tuttavia, nel caso di Syncope, il miglior compromesso complessivo resta RandomForest con **ClassBalancer**, senza selezione preventiva delle feature.

3.3 RQ3 – Quante classi buggy si sarebbero potute prevenire con zero smell?

La RQ3 stima quante classi predette come buggy nello scenario reale non sarebbero più state predette tali in uno scenario controfattuale con $NSMELLS = 0$ e $SMELL_DENSITY = 0$. L'analisi usa RandomForest con **ClassBalancer**, selezionato nella Milestone 2 come migliore configurazione complessiva. I risultati complessivi sui dataset **A**, **B+**, **B** e **C** sono riportati nella Tabella 7.

Sul dataset originale **A**, il modello predice 1215 istanze buggy a fronte di 1155 istanze buggy reali. L'errore di base è quindi una sovrastima di 60 istanze, pari al 5.19%. Questo valore mostra che le stime successive devono essere interpretate come predizioni del modello, non come bug certamente presenti o certamente evitati.

La stima dei bug prevenibili si basa sul confronto tra **B+**, che contiene le istanze originariamente smelly, e **B**, ottenuto azzerando $NSMELLS$ e $SMELL_DENSITY$ sulle stesse istanze. Nel dataset **B+** il modello predice 693 istanze buggy, mentre nel dataset sintetico **B** le predizioni buggy scendono a 659. Il numero di istanze classe-release potenzialmente prevenibili è quindi:

$$693 - 659 = 34$$

La Figura 5 evidenzia che la riduzione delle predizioni buggy è presente ma limitata: da 693 a 659 istanze, con un delta pari a 34.

La riduzione corrisponde al 4.91% delle predizioni buggy nel sottoinsieme smelly, al 2.80% delle predizioni buggy totali del dataset originale e al 2.94% delle istanze buggy reali presenti in **A**. Il dataset **C**, già privo di smell, rappresenta invece la parte non modificabile dall'intervento simulato.

Il risultato indica un effetto misurabile ma limitato. Gli smell influenzano alcune predizioni di bugginess, ma non sono l'unico fattore rilevante: dimensione, complessità, churn, fix history e ownership continuano a contribuire alla classificazione. Il risultato va quindi letto come una variazione nelle predizioni del modello nello scenario simulato, non come una misura diretta dei bug che sarebbero stati effettivamente evitati.

3.4 RQ4 – Quanto è accurato ed efficace il refactoring automatico tramite LLM?

La RQ4 valuta se le varianti generate tramite LLM siano compilabili, riducano le issue statiche e preservino il comportamento osservabile di C_0 . Le classi analizzate sono `DefaultJobManager` e `AuthDataAccessor`; per ciascuna classe, C_0 indica la versione originale e C_1 – C_4 le varianti rifattorizzate. Le issue residue per macrocategoria e gli esiti complessivi sono riportati nelle Tabelle 9 e 10.

Tutte le varianti C_1 – C_4 risultano compilabili nell’esito finale, in alcuni casi dopo prompt di riparazione limitati a problemi compilativi. L’efficacia del refactoring varia però in base alla classe analizzata: `DefaultJobManager` passa da 7 issue in C_0 a 0 issue in C_1 , C_2 e C_3 , mentre C_4 mantiene 2 issue; per `AuthDataAccessor`, invece, non emerge un miglioramento statico complessivo, poiché C_0 , C_1 e C_4 presentano 8 issue, mentre C_2 e C_3 arrivano a 9.

La validazione tramite regression testing mostra che la riduzione delle issue non è sufficiente, da sola, per considerare efficace un refactoring. Per `DefaultJobManager`, solo C_1 preserva il comportamento osservabile della baseline nel perimetro testato, mentre C_2 , C_3 e C_4 modificano la gestione di `jobData = null`. Per `AuthDataAccessor`, invece, tutte le varianti superano le suite di regressione disponibili, ma non riducono realmente le issue.

La Tabella 11 riporta la variazione di alcune feature statiche ricalcolabili, escludendo metriche storiche ed evolutive come churn, ownership e numero di autori, poiché le varianti generate tramite LLM non hanno una storia Git reale. Le frecce indicano la direzione della variazione rispetto a C_0 .

La risposta alla research question è quindi solo parzialmente positiva: l’LLM produce varianti compilabili e, in alcuni casi, compatibili con la baseline, ma non garantisce contemporaneamente riduzione delle issue e preservazione del comportamento. Per questo motivo, il refactoring automatico va interpretato come supporto alla manutenzione, non come sostituto della validazione ingegneristica.

4 Minacce alla validità

4.1 Minacce relative alla Milestone 1

4.1.1 Release recenti e difetti dormienti

È stato considerato solo il primo 34% delle release, scartando il restante 66%, per ridurre il rischio di *snoring*. Questo fenomeno riguarda le classi delle release più recenti che potrebbero contenere difetti non ancora scoperti o non ancora risolti e che, di conseguenza, verrebbero etichettate erroneamente come `Buggy = no`. Questa scelta riduce il rischio di falsi negativi nella label `Buggy`, ma limita la quantità di dati e la copertura della storia evolutiva di Syncope. Il criterio è stato applicato in modo uniforme, accettando il compromesso tra numero di istanze disponibili e affidabilità della ground truth.

4.1.2 Ricostruzione delle versioni affette

La label `Buggy` dipende dalla `ComputedAV`; affected version mancanti o imprecise nei ticket Jira possono quindi introdurre rumore nella ground truth. L’uso di Proportion Total permette di aumentare la copertura del dataset senza scartare i ticket incompleti, ma introduce una stima approssimata dell’injected version basata sui ticket meglio documentati.

Questa scelta introduce una possibile minaccia di *Time-Travel Data Leakage*: calcolare il Proportion sull’intero progetto e applicarlo anche alle release precedenti può implicare l’uso di informazioni disponibili solo in momenti successivi della storia evolutiva. Tale rischio è stato accettato come trade-off metodologico, poiché consente di non scartare un numero eccessivo di ticket privi di affected version esplicita e di mantenere una maggiore copertura del dataset.

La minaccia è stata mitigata applicando regole uniformi e riproducibili per validare o stimare le versioni affette.

4.1.3 SZZ e linkage tra Jira e Git

L'identificazione delle classi buggy dipende dal collegamento tra ticket, fix commit e classi modificate. La procedura ispirata a SZZ può essere sensibile a refactoring, rename, aggiunte di codice e commit non esplicitamente collegati al ticket, introducendo possibili falsi negativi.

La minaccia è stata mitigata combinando le classi individuate con la finestra temporale della `ComputedAV` e con l'inventario reale delle release.

4.1.4 Smell, classi di test

La misurazione degli smell dipende dal ruleset PMD adottato; di conseguenza, `NSMELLS` e `SMELL_DENSITY` non rappresentano tutti i possibili problemi di manutenibilità, ma solo le violation rilevate dallo strumento.

Il dataset include solo classi Java di produzione, escludendo le classi di test. Questa scelta è coerente con l'obiettivo di defect prediction sulle classi applicative, ma può escludere informazioni indirette sulla qualità del progetto.

4.2 Minacce relative alla Milestone 2

4.2.1 Sbilanciamento e balancing

Il dataset è fortemente sbilanciato verso la classe non buggy. Questa condizione può favorire modelli apparentemente accurati ma poco utili nella defect prediction, perché orientati a predire soprattutto la classe maggioritaria `no`.

La minaccia è stata gestita tramite `ClassBalancer`, che riequilibra il contributo delle classi modificando i pesi delle istanze, senza applicare esplicitamente oversampling o undersampling. Questa scelta evita sia la duplicazione artificiale di istanze buggy sia la rimozione di istanze non buggy, ma può comunque aumentare i falsi positivi e ridurre la Precision.

Per questo motivo, il balancing è stato interpretato come un compromesso, e i modelli sono stati valutati con metriche più informative della sola accuracy: Precision, Recall, AUC e Kappa.

4.2.2 Feature selection

La feature selection è stata eseguita con `InfoGainAttributeEval` e `Ranker`. Questa scelta introduce una possibile minaccia perché le feature vengono valutate singolarmente rispetto alla classe target, senza considerare direttamente l'interazione con il classificatore utilizzato. Di conseguenza, una feature poco informativa da sola ma utile in combinazione con altre potrebbe essere penalizzata o esclusa.

La minaccia è stata mitigata valutando più soglie di selezione, cioè top 5, top 10 e top 15 feature, e confrontando poi le prestazioni dei classificatori sui diversi sottoinsiemi ottenuti.

4.2.3 Classificatori e validazione

Il confronto è stato limitato a `RandomForest`, `NaiveBayes` e `IBk`. Questa scelta copre famiglie diverse di modelli, ma non esclude che altri classificatori possano ottenere prestazioni migliori.

La validazione è stata eseguita con 10 times 10-fold cross-validation, che fornisce una stima stabile e confrontabile tra configurazioni, ma non preserva l'ordine temporale delle release. Di conseguenza, le prestazioni possono risultare più ottimistiche rispetto a uno scenario operativo, in cui il modello dovrebbe essere addestrato su release passate e applicato a release future.

4.3 Minacce relative alla Milestone 3

4.3.1 What-if analysis e propagazione dell'errore

La what-if analysis azzerava solo `NSMELLS` e `SMELL_DENSITY`, mantenendo invariate tutte le altre feature. Questa assunzione consente di isolare l'effetto degli smell sulle predizioni, ma non rappresenta completamente un refactoring reale: nella pratica, rimuovere smell potrebbe modificare anche dimensione, complessità, fan-out o metriche evolutive successive.

La stima dipende inoltre dal modello selezionato, cioè `RandomForest` con `ClassBalancer`. Un modello diverso avrebbe potuto mostrare una diversa sensibilità agli smell, mentre eventuali errori di classificazione vengono propagati anche allo scenario controfattuale. Per questo motivo, il risultato va interpretato come differenza relativa tra $\mathbf{B}+$ e $\mathbf{B}$, non come numero assoluto di bug effettivamente evitati.

4.3.2 Dipendenza dal dataset e generalizzabilità

La what-if analysis eredita le assunzioni della Milestone 1, incluse eventuali imprecisioni nel labeling, nella stima della Injected Version, nel linkage tra ticket e commit e nel calcolo degli smell. Tali errori possono propagarsi al modello e influenzare la stima delle istanze potenzialmente prevenibili.

La mitigazione consiste nei controlli di coerenza applicati alla pipeline e nel limitare l'interpretazione dei risultati al caso di studio analizzato.

4.4 Minacce relative alla Milestone 4

4.4.1 Selezione delle classi e misura delle issue statiche

La selezione iniziale delle classi dipende dal calcolo di `NSMELLS` e dai criteri di filtraggio adottati. Un diverso ruleset PMD, un altro strumento di analisi statica o soglie diverse su LOC, numero di metodi e complessità ciclomatica avrebbero potuto produrre un insieme di classi differente. Inoltre, l'analisi di sole due classi limita la generalizzabilità dei risultati.

La minaccia è stata mitigata applicando criteri espliciti e riproducibili. Le soglie sono state usate per escludere classi banali o poco adatte alla validazione, selezionando classi concrete, compilabili e sufficientemente articolate per valutare il refactoring automatico.

4.4.2 Non determinismo e validazione delle varianti

Le varianti C_1 – C_4 sono state generate tramite LLM a partire da prompt testuali e quantità crescenti di contesto. Questa scelta introduce una minaccia alla replicabilità, perché una nuova esecuzione potrebbe produrre varianti diverse anche a parità di classe e prompt.

Il limite principale riguarda la correttezza semantica: l'LLM può introdurre modifiche apparentemente migliorative, ma non equivalenti alla baseline. La minaccia è stata mitigata documentando prompt, contesto e varianti ottenute, e valutando gli output effettivamente generati, senza selezionare retroattivamente solo quelli più favorevoli.

4.4.3 Possibile training set leakage dell'LLM

Una minaccia specifica dell'uso di LLM riguarda il possibile *Training Set Leakage*. Poiché Microsoft Copilot è addestrato anche su repository open-source disponibili su piattaforme come GitHub, è plausibile che il modello avesse già osservato durante il training parte del codice o classi simili appartenenti ad Apache Syncope. Questo data leakage potrebbe aver gonfiato artificialmente la capacità dell'LLM di produrre refactoring compilabili e semanticamente coerenti, riducendo la generalizzabilità dei risultati a progetti non presenti nel training set del modello.

4.4.4 Generalizzabilità dei risultati

Poiché l'analisi è stata condotta su due sole classi di Apache Syncope, i risultati non sono direttamente generalizzabili a tutte le classi del progetto, ad altri sistemi software o ad altri LLM. Classi con dimensione, responsabilità o tipologie di issue diverse potrebbero portare a esiti differenti.

La Milestone 4 non dimostra che il refactoring automatico sia sempre efficace, ma evidenzia la necessità di validare le varianti generate rispetto a compilazione, issue statiche e comportamento osservabile.

5 Conclusioni e takeaway messages

Il progetto ha mostrato che la qualità del dataset è un elemento centrale nella defect prediction. Errori nella ground truth, nel linkage tra ticket e commit o nella stima delle release affette possono propagarsi alle fasi successive, influenzando sia l'addestramento dei modelli sia l'interpretazione dei risultati.

Un secondo risultato riguarda la scelta del classificatore. In presenza di dati sbilanciati, la sola accuracy non è sufficiente: metriche come Precision, Recall, AUC e Kappa sono necessarie per valutare realmente la capacità di individuare classi buggy. Nel caso di Syncope, la configurazione più efficace è risultata RandomForest con **ClassBalancer**, scelta poi come modello di riferimento per la what-if analysis.

Gli smell rappresentano un segnale utile ma non esaustivo. La what-if analysis mostra che l'azzeramento delle feature di smell produce una riduzione misurabile ma limitata delle predizioni buggy. Questo conferma che la bugginess dipende anche da dimensione, complessità, churn, fix history e ownership, e non può essere spiegata esclusivamente dalla presenza di smell.

Infine, il refactoring automatico tramite LLM si è dimostrato promettente ma non autonomamente affidabile. Le varianti generate possono compilare e ridurre alcune issue statiche, ma non garantiscono sempre la preservazione del comportamento osservabile. Per questo motivo, modelli predittivi e LLM devono essere interpretati come strumenti di supporto alla manutenzione software, da affiancare sempre a validazione ingegneristica, metriche, test e analisi critica dei risultati.

Tabelle e figure

Tabella 1: Metriche raccolte nella Milestone 1

Metrica	Finestra	Significato
SIZE_LOC	R_i	Numero di linee della classe nella release considerata.
NOM	R_i	Numero di metodi dichiarati nella classe.
AVG_METHOD_SIZE	R_i	Dimensione media dei metodi, calcolata come $SIZE_LOC / NOM$.
CYCLO_COMPLEXITY	R_i	Complessità ciclomatica approssimata della classe.
FAN_OUT	R_i	Numero di dipendenze interne importate dalla classe.
NR	$F_0 \rightarrow R_i$	Numero di commit che hanno modificato la classe.
NFIX	$F_0 \rightarrow R_i$	Numero di fix commit che hanno modificato la classe.
NAUTH	$F_0 \rightarrow R_i$	Numero di autori distinti che hanno modificato la classe.
LOC_ADDED	$F_0 \rightarrow R_i$	Somma delle linee aggiunte alla classe.
MAX_LOC_ADDED	$F_0 \rightarrow R_i$	Massimo numero di linee aggiunte in un singolo commit.
CHURN	$F_0 \rightarrow R_i$	Somma delle linee aggiunte e cancellate nei commit della classe.
MAX_CHURN	$F_0 \rightarrow R_i$	Massimo valore di churn osservato in un singolo commit.
MAX_CHANGE_SET_SIZE	$F_0 \rightarrow R_i$	Massimo numero di file modificati insieme alla classe.
AVG_MODIFIED_DIRS	$F_0 \rightarrow R_i$	Numero medio di directory modificate nei commit della classe.
AGE_SINCE_LAST_CHANGE	R_i	Giorni tra l'ultima modifica della classe e la data della release.
OWNERSHIP_RATIO	$F_0 \rightarrow R_i$	Quota di commit attribuibile all'autore principale della classe.
SAME_DIRECTORY_CHANGE_RATIO	$F_0 \rightarrow R_i$	Quota di commit che modificano solo file nella stessa directory della classe.
AGE	$F_0 \rightarrow R_i$	Giorni tra la prima modifica storica della classe e la data della release.
SMELL_DENSITY	R_i	Numero di smell normalizzato rispetto alla dimensione della classe.
NSMELLS	R_i	Numero di smell rilevati da PMD sulla classe.
<i>Legenda finestre:</i> R_i indica lo snapshot della release corrente; $F_0 \rightarrow R_i$ indica la storia del file dalla sua prima comparsa nel repository fino alla release corrente.		

Tabella 2: Distribuzione delle istanze classe-release e della bugginess per release

Indice	Release	Classi	Buggy	Non buggy	% buggy
1	1.0.0-incubating	488	3	485	0.61%
2	1.0.1-incubating	489	2	487	0.41%
3	1.0.3-incubating	491	7	484	1.43%
4	1.0.2-incubating	491	6	485	1.22%
5	1.0.4	508	26	482	5.12%
6	1.0.5	590	125	465	21.19%
7	1.0.6	624	100	524	16.03%
8	1.0.7	638	51	587	7.99%
9	1.1.0	645	63	582	9.77%
10	1.0.8	645	58	587	8.99%
11	1.1.1	646	59	587	9.13%
12	1.1.2	650	56	594	8.62%
13	1.1.3	654	84	570	12.84%
14	1.1.4	641	99	542	15.44%
15	1.1.5	689	88	601	12.77%
16	1.1.6	722	13	709	1.80%
17	1.1.7	724	49	675	6.77%
18	1.1.8	787	25	762	3.18%
19	1.2.0-M1	795	16	779	2.01%
20	1.2.0	812	35	777	4.31%
21	1.2.1	818	18	800	2.20%
22	1.2.2	868	10	858	1.15%
23	1.2.3	972	12	960	1.23%
24	1.2.4	1080	25	1055	2.31%
25	1.2.5	1173	32	1141	2.73%
26	1.2.6	1462	39	1423	2.67%
27	2.0.0-M1	1310	54	1256	4.12%

Nota: l'indice segue l'ordine cronologico delle release usato nella pipeline, non l'ordinamento semantico del nome della versione.

Tabella 3: Indicatori descrittivi su smell e linkage Jira-Git

Indicatore	Valore
Istanze con almeno uno smell	7096
Istanze senza smell	13316
Massimo numero di smell su una istanza	44
Ticket con almeno un fix commit	812 / 877
Ticket-to-fix linkage	92.59%

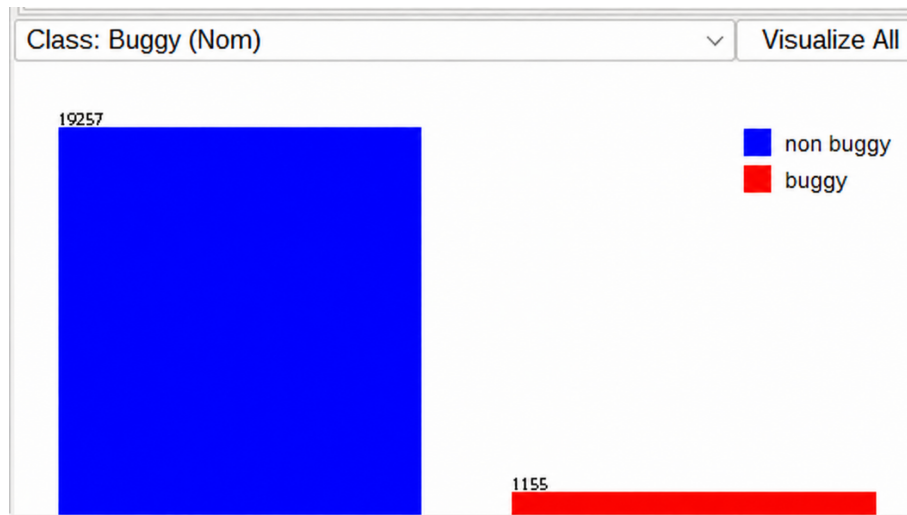


Figura 1: Distribuzione delle istanze buggy e non buggy nel dataset finale.

Tabella 4: Confronto descrittivo tra istanze buggy e non buggy

Metrica	Mediana non buggy	Mediana buggy
SIZE_LOC	74	173
CYCLO_COMPLEXITY	5	12
CHURN	246	632
AGE_SINCE_LAST_CHANGE	83	36

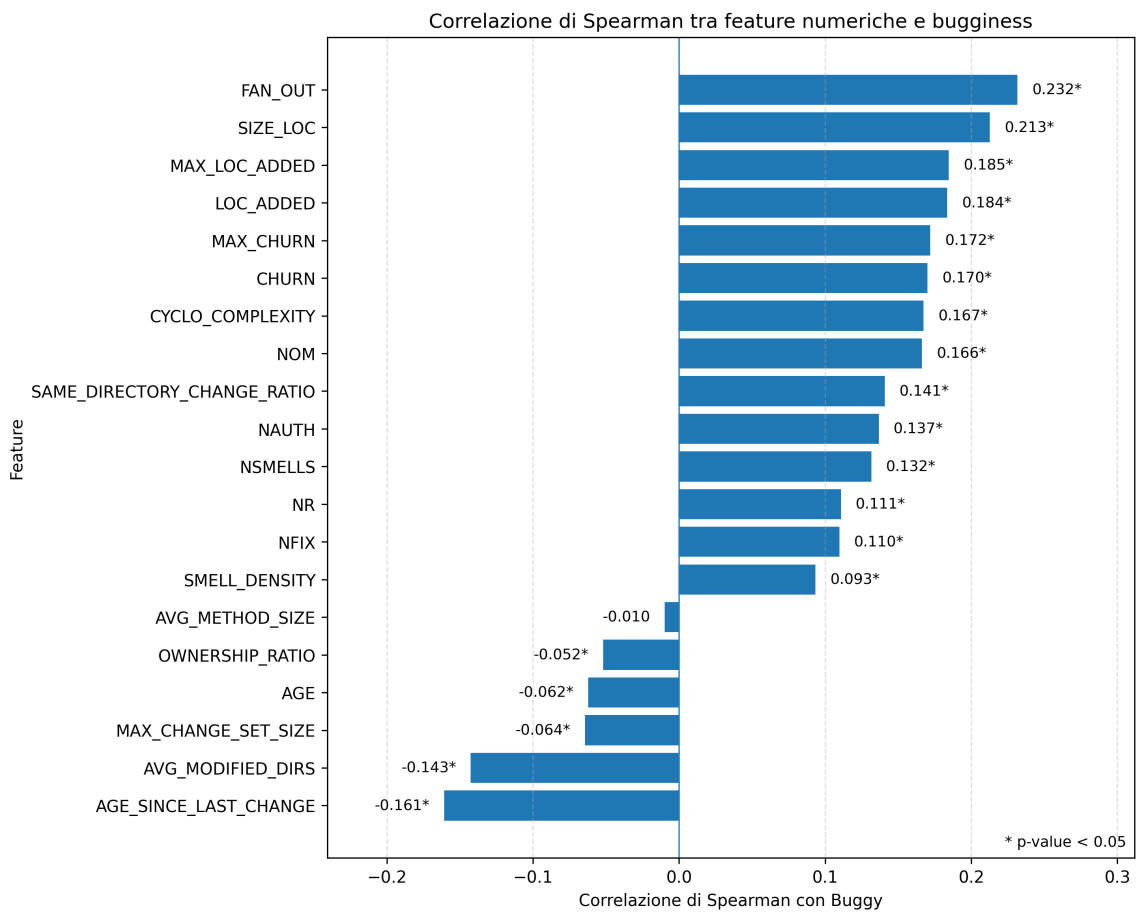


Figura 2: Correlazione di Spearman tra le feature numeriche e la label di bugginess. L'asterisco indica $p\text{-value} < 0.05$.

Tabella 5: Risultati delle configurazioni sperimentali della Milestone 2

Configurazione	Classifier	Precision	Recall	AUC	Kappa
No filter	RandomForest	0.793	0.425	0.915	0.534
No filter	NaiveBayes	0.194	0.390	0.783	0.198
No filter	IBk	0.584	0.493	0.742	0.508
Balancing only	RandomForest	0.611	0.526	0.914	0.540
Balancing only	NaiveBayes	0.188	0.476	0.783	0.205
Balancing only	IBk	0.584	0.493	0.742	0.508
Feature selection only – top 10	RandomForest	0.786	0.412	0.902	0.520
Feature selection only – top 10	NaiveBayes	0.191	0.350	0.774	0.187
Feature selection only – top 10	IBk	0.424	0.408	0.695	0.381
FS + balancing – top 5	RandomForest	0.438	0.507	0.873	0.433
FS + balancing – top 5	NaiveBayes	0.184	0.409	0.772	0.190
FS + balancing – top 5	IBk	0.407	0.472	0.788	0.400
FS + balancing – top 10	RandomForest	0.564	0.512	0.902	0.509
FS + balancing – top 10	NaiveBayes	0.175	0.417	0.774	0.181
FS + balancing – top 10	IBk	0.424	0.408	0.695	0.381
FS + balancing – top 15	RandomForest	0.588	0.513	0.908	0.522
FS + balancing – top 15	NaiveBayes	0.180	0.454	0.777	0.192
FS + balancing – top 15	IBk	0.551	0.472	0.731	0.481

Tabella 6: Migliori configurazioni per metrica

Metrica	Migliore configurazione	Valore
Precision	RandomForest, no filter	0.793
Recall	RandomForest, balancing only	0.526
AUC	RandomForest, no filter	0.915
Kappa	RandomForest, balancing only	0.540

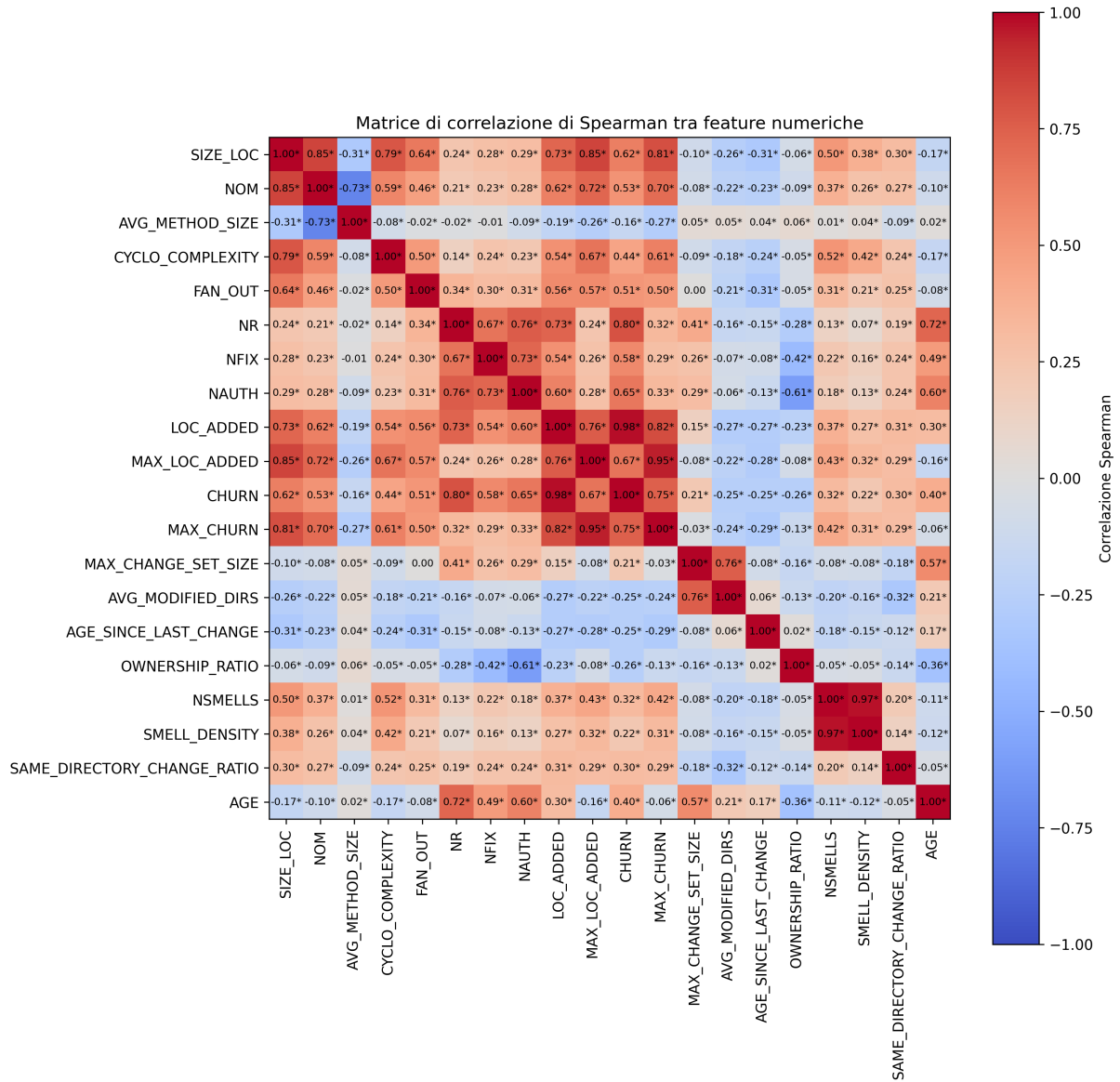


Figura 3: Matrice di correlazione di Spearman tra le feature numeriche del dataset.

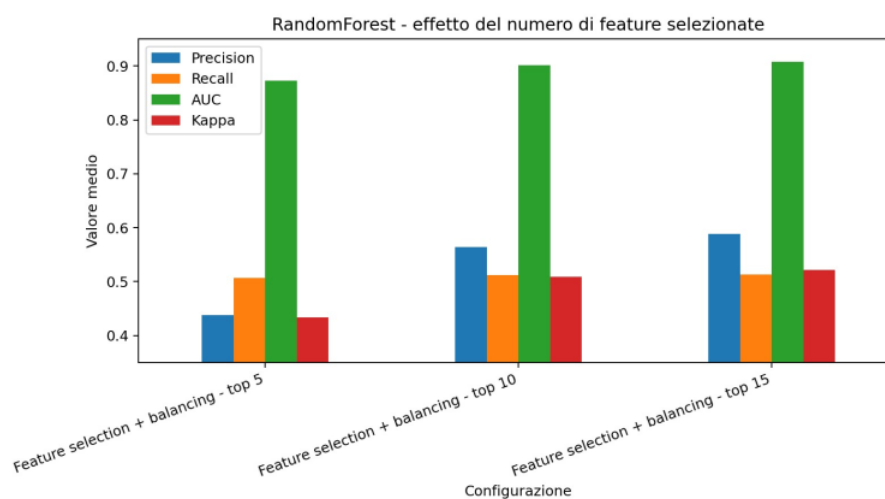


Figura 4: Effetto del numero di feature selezionate sulle prestazioni di RandomForest con feature selection e balancing.

Tabella 7: Dataset utilizzati nella what-if analysis: classi buggy reali e stimate

Dataset	Istanze	NSMELLS > 0	NSMELLS = 0	Buggy reali	Non buggy reali	Buggy stimate
A	20412	7096	13316	1155	19257	1215
B+	7096	7096	0	649	6447	693
B	7096	0	7096	649	6447	659
C	13316	0	13316	506	12810	522

Tabella 8: Confusion matrix aggregate della what-if analysis

Dataset	Pred. buggy	TN	FP	FN	TP
A	1215	19197	60	0	1155
B+	693	6403	44	0	649
B	659	6424	23	13	636
C	522	12794	16	0	506

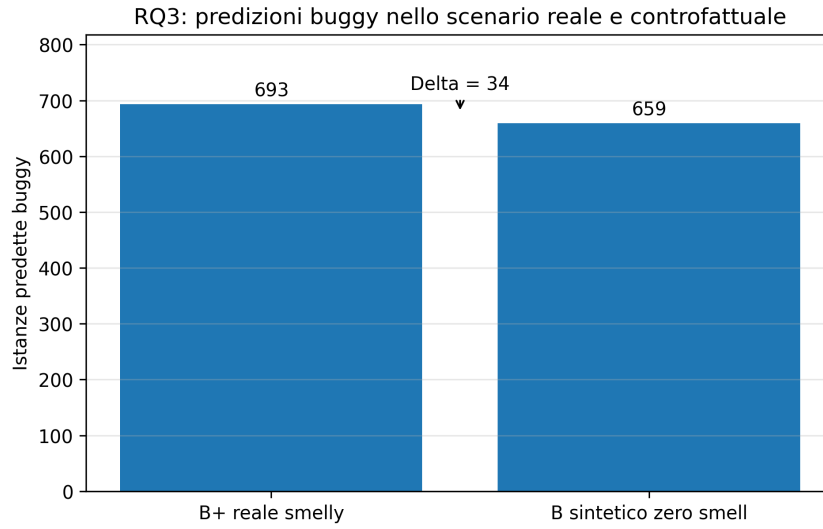


Figura 5: Predizioni buggy nello scenario reale smelly e nello scenario controfattuale con zero smell.

Listato 1– Prompt base per la generazione delle varianti rifattorizzate

Prompt base per il refactoring automatico

You are an expert Java developer.

I want to improve the maintainability of the following Java class from Apache Syncope.

Create a refactored version of the class without changing its external behavior.
The refactored class must replace the original class in the existing system.

Strict constraints:

- Keep the same package declaration.
- Keep the same class name: <CLASS_NAME>.
- Keep the same public API.
- Keep all public and protected method signatures unchanged.
- Do not change the observable behavior.
- Do not modify or require changes to other classes.
- Do not introduce unrelated changes.
- Do not remove required annotations, dependency injections, logging, or framework-specific behavior.
- The class must compile inside the existing Apache Syncope project.
- Return the complete Java class only, with no markdown and no explanation.

Refactoring goal:

- Improve maintainability.
- Reduce the SonarCloud code smells listed below.
- Prefer small, behavior-preserving refactorings.
- Extract helper methods only when this improves readability.
- Reduce duplication where possible.
- Keep the logic easy to compare with the original version.

SonarCloud smells to address:

<SONARCLOUD_DIAGNOSTIC>

Additional regression constraints:

<OPTIONAL_TESTS_FOR_C2_C3_C4>

Original class:

<ORIGINAL_CLASS>

Tabella 9: Macrocategorie di issue statiche residue nelle varianti refactorizzate

Classe	Categoria	C_0	C_1	C_2	C_3	C_4
DefaultJobManager	Complessità e struttura	3	0	0	0	0
DefaultJobManager	Logging e naming	1	0	0	0	0
DefaultJobManager	Concorrenza/sincronizzazione	2	0	0	0	2
DefaultJobManager	Parametri del costruttore	1	0	0	0	0
AuthDataAccessor	Complessità e struttura	4	4	5	5	4
AuthDataAccessor	Naming e metodi vuoti	0	4	4	4	4
AuthDataAccessor	Logging/transazioni	2	0	0	0	0
AuthDataAccessor	Parametri del costruttore	1	1	1	1	1
AuthDataAccessor	Espressioni booleane	1	1	1	1	1
Totale DefaultJobManager	–	7	0	0	0	2
Totale AuthDataAccessor	–	8	8	9	9	8

Tabella 10: Esito complessivo delle varianti refactorizzate

Classe	Variante	Compila	Issue	Regressione	Osservazione
DefaultJobManager	C_0	Si	7	Pass	Baseline originale.
DefaultJobManager	C_1	Si	0	Pass	Smell rimossi e comportamento preservato.
DefaultJobManager	C_2	Si	0	Fail	Smell rimossi, ma gestione diversa del caso <code>jobData = null</code> .
DefaultJobManager	C_3	Si	0	Fail	Smell rimossi, ma gestione diversa del caso <code>jobData = null</code> .
DefaultJobManager	C_4	Si	2	Fail	Issue ridotte, ma gestione diversa del caso <code>jobData = null</code> .
AuthDataAccessor	C_0	Si	8	Pass	Baseline originale.
AuthDataAccessor	C_1	Si	8	Pass	Nessuna regressione osservata, ma issue non ridotte.
AuthDataAccessor	C_2	Si	9	Pass	Nessuna regressione osservata, ma issue aumentate.
AuthDataAccessor	C_3	Si	9	Pass	Nessuna regressione osservata, ma issue aumentate.
AuthDataAccessor	C_4	Si	8	Pass	Nessuna regressione osservata, ma issue non ridotte.

Tabella 11: Variazione delle feature statiche rispetto a C_0

Classe	Var.	SIZE_LOC	NOM	AVG_METHOD_SIZE	CYCLO	FAN_OUT
DefaultJobManager	C_1	↑	↑	↓	→	↑
DefaultJobManager	C_2	↑	↑	↓	→	↑
DefaultJobManager	C_3	↑	↑	↓	→	↑
DefaultJobManager	C_4	↑	↑	↓	↑	→
AuthDataAccessor	C_1	↑	↑	↓	→	→
AuthDataAccessor	C_2	→	→	→	→	→
AuthDataAccessor	C_3	↑	↑	↓	→	→
AuthDataAccessor	C_4	↑	↑	↓	→	→
AuthDataAccessor	C_4	↑	↑	↓	→	→

Nota: le frecce indicano la variazione rispetto a C_0 ; ↑ su feature positivamente associate alla bugginess può suggerire un peggioramento strutturale. La freccia → indica una variazione nulla o trascurabile nelle feature statiche considerate.